Message sprint 3 & 4 thread search AI Admin

MESSAGING SPRINT 3 — THREADS + SEARCH + FILES + NOTIFICATIONS

PREREQUISITE: Messaging Sprints 1-2 complete

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD

1. Thread System (reply in thread, thread panel, thread following, "Also send to channel")
2. Message Search (full-text, filters, search UI)
3. File Sharing (upload, inline previews, file browser)
4. Notification System (push, in-app, per-channel prefs)

FEATURE: THREADS

Thread Panel UI: `src/components/messaging/thread-panel.tsx`

Build a 380px slide-in panel on the right side:

```
┌──────────────────────────────────────┐
│ Thread in #engineering    [×]        │
├──────────────────────────────────────┤
│ [Parent message — full display]      │
```

```
│ ─── 5 replies ──────────────────────── │
│ [Reply 1]                              │
│ [Reply 2]                              │
│ [Reply 3 with attachment]              │
│ [Reply 4]                              │
│ [Reply 5]                              │
│                                        │
│ ┌──────────────────────────────────┐   │
│ │ □ Also send to #engineering      │   │
│ │ [Reply composer____________]     │   │
│ │                        [Send]    │   │
│ └──────────────────────────────────┘   │
└────────────────────────────────────────┘
```

Behavior:

- Click "Reply in thread" on any message → opens thread panel
- Thread replies use the same message_send socket event with `threadParentId`
- "Also send to #channel" checkbox: If checked, message posted both in thread AND main channel view
- Thread following: Auto-follow when you reply. Unfollow button in panel header.
- Sidebar "Threads" section: Shows all threads user follows that have new replies. Badge count.
- 10+ replies → "🤖 Summarize Thread" button at top (uses AI from Feature 11)
- Thread participant avatars shown as row below parent message

Thread tRPC additions (add to existing messages.ts router):

```
// Add to src/server/routers/messages.ts

  // Get threads user is following with new replies
  getFollowedThreads: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
```

```
    .query(async ({ input, ctx }) => {
      // Find messages where user is in threadParticipantIds
and has new replies
      const threads = await prisma.message.findMany({
        where: {
          channel: { workspaceId: input.workspaceId },
          threadReplyCount: { gt: 0 },
          threadParticipantIds: { has: ctx.user.id },
          deletedAt: null,
        },
        orderBy: { threadLastReplyAt: "desc" },
        take: 20,
        select: {
          id: true, channelId: true, content: true, authorId:
true,
          threadReplyCount: true, threadLastReplyAt: true, th
readParticipantIds: true,
          channel: { select: { displayName: true } },
        },
      });
      return threads;
    }),

  // Get thread replies
  getThreadReplies: protectedProcedure
    .input(z.object({
      threadParentId: z.string().uuid(),
      limit: z.number().default(50),
      cursor: z.string().uuid().optional(),
    }))
    .query(async ({ input }) => {
      const replies = await prisma.message.findMany({
        where: { threadParentId: input.threadParentId, delete
dAt: null },
        orderBy: { createdAt: "asc" },
        take: input.limit,
```

```
        include: { attachments: true },
      });
      return replies;
    }),
```

FEATURE: MESSAGE SEARCH

tRPC Router: `src/server/routers/message-search.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const messageSearchRouter = router({
  search: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      query: z.string().min(1).max(500),
      filters: z.object({
        fromUserId: z.string().uuid().optional(),
        inChannelId: z.string().uuid().optional(),
        hasAttachment: z.boolean().optional(),
        beforeDate: z.string().optional(),
        afterDate: z.string().optional(),
        isPinned: z.boolean().optional(),
      }).optional(),
      limit: z.number().default(20),
      offset: z.number().default(0),
    }))
    .query(async ({ input, ctx }) => {
      // Get user's channel memberships for permission filter
ing
      const memberships = await prisma.channelMember.findMany
({
```

```
        where: { userId: ctx.user.id },
        select: { channelId: true },
      });
      const accessibleChannelIds = memberships.map(m => m.cha
nnelId);

      const where: any = {
        channel: { workspaceId: input.workspaceId },
        channelId: { in: accessibleChannelIds },
        deletedAt: null,
      };

      if (input.filters?.fromUserId) where.authorId = input.f
ilters.fromUserId;
      if (input.filters?.inChannelId) where.channelId = inpu
t.filters.inChannelId;
      if (input.filters?.hasAttachment) where.hasAttachments
= true;
      if (input.filters?.isPinned) where.isPinned = true;
      if (input.filters?.beforeDate) where.createdAt = { ...w
here.createdAt, lte: new Date(input.filters.beforeDate) };
      if (input.filters?.afterDate) where.createdAt = { ...wh
ere.createdAt, gte: new Date(input.filters.afterDate) };

      // Full-text search using PostgreSQL
      const results = await prisma.$queryRaw<Array<{
        id: string; channelId: string; authorId: string; cont
ent: string;
        createdAt: Date; channelName: string;
      }>>`
        SELECT m.id, m."channelId", m."authorId",
          ts_headline('english', m.content, plainto_tsquery
('english', ${input.query}),
            'StartSel=<mark>, StopSel=</mark>, MaxFragments=
2, MaxWords=30') as content,
          m."createdAt",
```

```
          c."displayName" as "channelName"
        FROM "Message" m
        JOIN "Channel" c ON m."channelId" = c.id
        WHERE c."workspaceId" = ${input.workspaceId}::uuid
          AND m."channelId" = ANY(${accessibleChannelIds}::uu
id[])
          AND m."deletedAt" IS NULL
          AND to_tsvector('english', m.content) @@ plainto_ts
query('english', ${input.query})
          ${input.filters?.fromUserId ? prisma.$queryRaw`AND
m."authorId" = ${input.filters.fromUserId}::uuid` : prisma.$q
ueryRaw``}
          ${input.filters?.inChannelId ? prisma.$queryRaw`AND
m."channelId" = ${input.filters.inChannelId}::uuid` : prism
a.$queryRaw``}
        ORDER BY ts_rank(to_tsvector('english', m.content), p
lainto_tsquery('english', ${input.query})) DESC
        LIMIT ${input.limit}
        OFFSET ${input.offset}
      `;

      return { results, hasMore: results.length === input.lim
it };
    }),
});
```

Search UI: `src/components/messaging/message-search.tsx`

Build a search modal (Ctrl+Shift+F within messaging):

- Input field with instant results
- Filter pills: [From: Anyone ▾] [In: Any channel ▾] [Has: attachment ▾] [Date range]
- Results show: channel name, author, highlighted snippet, timestamp

- Click result → navigates to that message in the channel with highlight animation
- Search operators in the input: `from:alice in:#engineering after:2026-03-01 has:file`

FEATURE: FILE SHARING

File Upload API: `src/app/api/files/upload/route.ts`

```
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/auth";
import { S3Client, PutObjectCommand } from "@aws-sdk/client-s
3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";
import { prisma } from "@/lib/prisma";

const s3 = new S3Client({
  region: process.env.S3_REGION || "auto",
  endpoint: process.env.S3_ENDPOINT,
  credentials: {
    accessKeyId: process.env.S3_ACCESS_KEY!,
    secretAccessKey: process.env.S3_SECRET_KEY!,
  },
});

const MAX_FILE_SIZE = 25 * 1024 * 1024; // 25MB

export async function POST(req: NextRequest) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.id) return NextResponse.json({ error:
"Unauthorized" }, { status: 401 });

  const formData = await req.formData();
  const file = formData.get("file") as File;
  const messageId = formData.get("messageId") as string;
```

```
  const messageType = formData.get("messageType") as string |
| "channel"; // 'channel' or 'dm'

  if (!file) return NextResponse.json({ error: "No file provi
ded" }, { status: 400 });
  if (file.size > MAX_FILE_SIZE) return NextResponse.json({ e
rror: "File too large (max 25MB)" }, { status: 400 });

  const key = `uploads/${session.user.id}/${Date.now()}-${fil
e.name}`;
  const buffer = Buffer.from(await file.arrayBuffer());

  await s3.send(new PutObjectCommand({
    Bucket: process.env.S3_BUCKET!,
    Key: key,
    Body: buffer,
    ContentType: file.type,
  }));

  const fileUrl = `${process.env.S3_PUBLIC_URL}/${key}`;

  // Generate thumbnail for images
  let thumbnailUrl: string | undefined;
  if (file.type.startsWith("image/")) {
    thumbnailUrl = fileUrl; // Use original for now; add shar
p resize in production
  }

  // Create attachment record
  if (messageType === "channel") {
    await prisma.messageAttachment.create({
      data: {
        messageId,
        fileName: file.name,
        fileUrl,
        fileSize: file.size,
```

```
        mimeType: file.type,
        thumbnailUrl,
      },
    });
    await prisma.message.update({ where: { id: messageId }, d
ata: { hasAttachments: true } });
  } else {
    await prisma.dmAttachment.create({
      data: {
        messageId,
        fileName: file.name,
        fileUrl,
        fileSize: file.size,
        mimeType: file.type,
        thumbnailUrl,
      },
    });
    await prisma.directMessage.update({ where: { id: messageI
d }, data: { hasAttachments: true } });
  }

  return NextResponse.json({ fileUrl, thumbnailUrl, fileName:
file.name, fileSize: file.size });
}
```

File Preview Component: `src/components/messaging/file-preview.tsx`

Build inline previews based on MIME type:

- **Images** (image/*): Max 400px width, click → lightbox with zoom/download
- **Videos** (video/*): Inline player with thumbnail, play button
- **Audio** (audio/*): Waveform player with play/pause
- **PDFs** (application/pdf): First page thumbnail + page count badge

- **Code** (.js, .ts, .py, .md, etc.): Syntax-highlighted first 20 lines + "Expand" button
- **Other**: File icon + name + size + download button

File Browser (add to channel detail panel):

List all files shared in the channel with:

- Grid view (thumbnails) / List view (names)
- Filter by type: Images | Documents | Code | Other
- Filter by user
- Sort by: Date | Name | Size

FEATURE: NOTIFICATIONS

Notification Service: `src/server/services/notification-service.ts`

```
import { prisma } from "@/lib/prisma";
import { getIO } from "@/server/services/socket-server";

interface NotifyParams {
  userId: string;
  workspaceId: string;
  type: string; // 'mention','dm','thread_reply','channel_act
ivity','task_mention'
  title: string;
  body: string;
  sourceType: string;
  sourceId: string;
  channelId?: string;
}

class NotificationService {
  async notify(params: NotifyParams) {
```

```
    // Check user's preferences
    const member = params.channelId
      ? await prisma.channelMember.findUnique({
          where: { channelId_userId: { channelId: params.chan
nelId, userId: params.userId } },
        })
      : null;

    // Skip if channel is muted and it's not a direct mention
    if (member?.isMuted && params.type !== "mention") return;

    // Skip if notification pref is "nothing"
    if (member?.notificationPref === "nothing") return;

    // Skip if pref is "mentions_only" and this isn't a menti
on
    if (member?.notificationPref === "mentions_only" && param
s.type !== "mention" && params.type !== "dm") return;

    // Check if user is in focus mode (Feature 9)
    const focusSession = await prisma.focusSession.findFirst
({
      where: { userId: params.userId, status: "active" },
    });

    if (focusSession) {
      // Queue notification for after focus session
      await prisma.focusSession.update({
        where: { id: focusSession.id },
        data: { queuedNotificationCount: { increment: 1 } },
      });
      // Only deliver if it's urgent (direct mention)
      if (params.type !== "mention" && params.type !== "dm")
return;
    }
```

```
    // Create in-app notification
    await prisma.notificationItem.create({
      data: {
        userId: params.userId,
        workspaceId: params.workspaceId,
        type: params.type,
        title: params.title,
        body: params.body,
        sourceType: params.sourceType,
        sourceId: params.sourceId,
        aiPriority: params.type === "mention" || params.type
=== "dm" ? "important" : "normal",
      },
    });

    // Real-time: emit to user's socket
    try {
      const io = getIO();
      io.to(`user:${params.userId}`).emit("notification", {
        type: params.type,
        title: params.title,
        body: params.body,
        channelId: params.channelId,
        sourceId: params.sourceId,
      });
    } catch {}

    // TODO: Web Push notification (implement with Web Push A
PI + FCM)
  }

  // Notify all mentioned users in a message
  async notifyMentions(message: { id: string; channelId: stri
ng; authorId: string; content: string; mentionedUserIds: stri
ng[] }, workspaceId: string, authorName: string) {
    for (const userId of message.mentionedUserIds) {
```

```
      if (userId === message.authorId) continue; // don't not
ify self
      await this.notify({
        userId,
        workspaceId,
        type: "mention",
        title: `${authorName} mentioned you in #channel`,
        body: message.content.slice(0, 100),
        sourceType: "message",
        sourceId: message.id,
        channelId: message.channelId,
      });
    }
  }

  // Notify thread followers
  async notifyThreadReply(threadParentId: string, replyAuthor
Id: string, replyContent: string, channelId: string, workspac
eId: string, authorName: string) {
    const parent = await prisma.message.findUnique({
      where: { id: threadParentId },
      select: { threadParticipantIds: true, authorId: true },
    });
    if (!parent) return;

    const toNotify = [...new Set([parent.authorId, ...parent.
threadParticipantIds])].filter(id => id !== replyAuthorId);

    for (const userId of toNotify) {
      await this.notify({
        userId,
        workspaceId,
        type: "thread_reply",
        title: `${authorName} replied in a thread`,
        body: replyContent.slice(0, 100),
        sourceType: "message",
```

```
        sourceId: threadParentId,
        channelId,
      });
    }
  }
}

export const notificationService = new NotificationService();
```

VERIFICATION — MESSAGING SPRINT 3

```
☐ Click "Reply in thread" → thread panel opens on right
☐ Send thread reply → appears in panel + parent shows "1 repl
y"
☐ Check "Also send to channel" → reply appears in both thread
and main channel
☐ Sidebar "Threads" section shows followed threads with new r
eply badges
☐ Thread with 10+ replies → "Summarize" button appears
☐ Search messages: type keyword → results appear with highlig
hted snippets
☐ Search with filter from:alice → only Alice's messages shown
☐ Search with filter in:#engineering → scoped to that channel
☐ Click search result → navigates to message location
☐ Upload file via 📎 button → uploads to S3 → preview shows i
nline
☐ Drag-and-drop file → same upload flow
☐ Paste screenshot (Ctrl+V) → image uploaded and previewed
☐ Image preview: click → lightbox with zoom
☐ PDF preview: shows first page thumbnail
☐ Code file: syntax-highlighted preview
☐ File browser in channel details → lists all shared files
☐ @mention in message → mentioned user receives notification
☐ Thread reply → all thread participants notified
```

```
□ Muted channel → no notifications except @mentions
□ User in focus mode → notifications queued (except mentions)
```

MESSAGING SPRINT 4 — HUDDLES + AI + ADMIN + POLISH

PREREQUISITE: Messaging Sprints 1-3 complete

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD

1. Voice Huddles (start, join, leave, screen share, live captions, AI summary)
2. AI Messaging Features (summarize, catch-up, smart replies, translate, decisions)
3. Admin Controls (channel management, retention, export, permissions)
4. Productivity Integrations (status sync, task mentions, meeting links)
5. 10 Innovative Features Beyond Slack
6. Polish (keyboard shortcuts, accessibility, mobile responsive)

FEATURE: VOICE HUDDLES

tRPC Router: `src/server/routers/huddles.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { getIO } from "@/server/services/socket-server";
```

```
import { aiService } from "@/lib/ai/ai-service";

export const huddlesRouter = router({
  // Start a huddle in a channel
  start: protectedProcedure
    .input(z.object({ channelId: z.string().uuid(), workspace
Id: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      // Check for existing active huddle
      const existing = await prisma.huddle.findFirst({
        where: { channelId: input.channelId, status: "active"
},
      });
      if (existing) throw new Error("A huddle is already acti
ve in this channel");

      const huddle = await prisma.huddle.create({
        data: {
          channelId: input.channelId,
          workspaceId: input.workspaceId,
          startedBy: ctx.user.id,
          participantIds: [ctx.user.id],
        },
      });

      // System message
      await prisma.message.create({
        data: {
          channelId: input.channelId,
          authorId: ctx.user.id,
          content: "started a huddle 🎧",
          type: "system",
          metadata: { huddleId: huddle.id },
        },
      });
```

```
      // Broadcast to channel
      const io = getIO();
      io.to(`channel:${input.channelId}`).emit("huddle_starte
d", {
        huddleId: huddle.id,
        channelId: input.channelId,
        startedBy: ctx.user.id,
      });

      return huddle;
    }),

  // Join an existing huddle
  join: protectedProcedure
    .input(z.object({ huddleId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const huddle = await prisma.huddle.findUnique({ where:
{ id: input.huddleId } });
      if (!huddle || huddle.status !== "active") throw new Er
ror("Huddle not found or ended");
      if (huddle.participantIds.length >= huddle.maxParticipa
nts) throw new Error("Huddle is full");

      await prisma.huddle.update({
        where: { id: input.huddleId },
        data: { participantIds: { push: ctx.user.id } },
      });

      const io = getIO();
      io.to(`channel:${huddle.channelId}`).emit("huddle_parti
cipant_joined", {
        huddleId: input.huddleId,
        userId: ctx.user.id,
      });

      return { success: true };
```

```
    }),

  // Leave huddle
  leave: protectedProcedure
    .input(z.object({ huddleId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const huddle = await prisma.huddle.findUnique({ where:
{ id: input.huddleId } });
      if (!huddle) throw new Error("Huddle not found");

      const updatedParticipants = huddle.participantIds.filte
r(id => id !== ctx.user.id);

      if (updatedParticipants.length === 0) {
        // Last person left — end huddle
        await prisma.huddle.update({
          where: { id: input.huddleId },
          data: { status: "ended", endedAt: new Date(), parti
cipantIds: [] },
        });

        // Generate AI summary if transcript exists
        if (huddle.transcript) {
          const summary = await aiService.generateText(
            `Summarize this huddle conversation:\n${huddle.tr
anscript}`,
            { systemPrompt: "Summarize huddle conversations.
Include key points and decisions.", maxTokens: 500, temperatu
re: 0.3 },
          );
          await prisma.huddle.update({
            where: { id: input.huddleId },
            data: { aiSummary: summary.text },
          });

          // Post summary as system message
```

```
          await prisma.message.create({
            data: {
              channelId: huddle.channelId,
              authorId: ctx.user.id,
              content: `🎧 Huddle ended\n\n**Summary:**\n${su
mmary.text}`,
              type: "system",
              metadata: { huddleId: huddle.id, huddleSummary:
true },
            },
          });
        }

        const io = getIO();
        io.to(`channel:${huddle.channelId}`).emit("huddle_end
ed", { huddleId: input.huddleId });
      } else {
        await prisma.huddle.update({
          where: { id: input.huddleId },
          data: { participantIds: updatedParticipants },
        });

        const io = getIO();
        io.to(`channel:${huddle.channelId}`).emit("huddle_par
ticipant_left", {
          huddleId: input.huddleId,
          userId: ctx.user.id,
        });
      }

      return { success: true };
    }),

  // Get active huddle for a channel
  getActive: protectedProcedure
    .input(z.object({ channelId: z.string().uuid() }))
```

```
    .query(async ({ input }) => {
      return prisma.huddle.findFirst({
        where: { channelId: input.channelId, status: "active"
},
      });
    }),
});
```

FEATURE: AI MESSAGING (uses AI service from Sprint 1)

The AI messaging router was already built in Sprint 7 of the AI Features (messagingAiRouter). It includes:

- Thread summarization
- Channel catch-up
- Smart reply suggestions
- AI compose
- Auto-translate
- Decision extraction

Ensure the messagingAiRouter is imported and merged in the root tRPC router.

FEATURE: ADMIN CONTROLS

tRPC Router: `src/server/routers/messaging-admin.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const messagingAdminRouter = router({
  // Get all channels with stats (admin view)
  getAllChannels: protectedProcedure
```

```
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.channel.findMany({
        where: { workspaceId: input.workspaceId, deletedAt: n
ull },
        orderBy: { messageCount: "desc" },
        select: {
          id: true, displayName: true, type: true, memberCoun
t: true,
          messageCount: true, lastMessageAt: true, isArchive
d: true, createdAt: true,
        },
      });
    }),

  // Set message retention policy
  setRetention: protectedProcedure
    .input(z.object({
      channelId: z.string().uuid(),
      retentionDays: z.number().min(1).max(3650).nullable(),
// null = keep forever
    }))
    .mutation(async ({ input }) => {
      return prisma.channel.update({
        where: { id: input.channelId },
        data: { retentionDays: input.retentionDays },
      });
    }),

  // Export channel messages (as JSON)
  exportChannel: protectedProcedure
    .input(z.object({
      channelId: z.string().uuid(),
      startDate: z.string().optional(),
      endDate: z.string().optional(),
    }))
```

```
    .mutation(async ({ input }) => {
      const messages = await prisma.message.findMany({
        where: {
          channelId: input.channelId,
          deletedAt: null,
          ...(input.startDate ? { createdAt: { gte: new Date
(input.startDate) } } : {}),
          ...(input.endDate ? { createdAt: { lte: new Date(in
put.endDate) } } : {}),
        },
        orderBy: { createdAt: "asc" },
        include: { attachments: true },
      });
      return { messages, count: messages.length };
    }),

  // Bulk manage channel members
  bulkManageMembers: protectedProcedure
    .input(z.object({
      channelId: z.string().uuid(),
      action: z.enum(["add", "remove", "change_role"]),
      userIds: z.array(z.string().uuid()),
      role: z.string().optional(),
    }))
    .mutation(async ({ input }) => {
      if (input.action === "add") {
        await prisma.channelMember.createMany({
          data: input.userIds.map(userId => ({ channelId: inp
ut.channelId, userId })),
          skipDuplicates: true,
        });
        await prisma.channel.update({
          where: { id: input.channelId },
          data: { memberCount: { increment: input.userIds.len
gth } },
        });
```

```
      } else if (input.action === "remove") {
        await prisma.channelMember.deleteMany({
          where: { channelId: input.channelId, userId: { in:
input.userIds } },
        });
        await prisma.channel.update({
          where: { id: input.channelId },
          data: { memberCount: { decrement: input.userIds.len
gth } },
        });
      } else if (input.action === "change_role" && input.rol
e) {
        await prisma.channelMember.updateMany({
          where: { channelId: input.channelId, userId: { in:
input.userIds } },
          data: { role: input.role },
        });
      }
      return { success: true };
    }),

  // Get messaging analytics
  getAnalytics: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      days: z.number().default(30),
    }))
    .query(async ({ input }) => {
      const since = new Date(Date.now() - input.days * 864000
00);

      const [totalMessages, activeChannels, activeUsers, topC
hannels] = await Promise.all([
        prisma.message.count({ where: { channel: { workspaceI
d: input.workspaceId }, createdAt: { gte: since } } }),
        prisma.channel.count({ where: { workspaceId: input.wo
```

```
rkspaceId, lastMessageAt: { gte: since } } }),
        prisma.message.groupBy({
          by: ["authorId"],
          where: { channel: { workspaceId: input.workspaceId
}, createdAt: { gte: since } },
        }).then(r => r.length),
        prisma.channel.findMany({
          where: { workspaceId: input.workspaceId },
          orderBy: { messageCount: "desc" },
          take: 10,
          select: { displayName: true, messageCount: true, me
mberCount: true },
        }),
      ]);

      return { totalMessages, activeChannels, activeUsers, to
pChannels, period: input.days };
    }),
});
```

---

10 INNOVATIVE FEATURES BEYOND SLACK

Build these as enhancements within existing components:

```
1. SMART CHANNEL SUGGESTIONS:
   When user sends a message, AI checks if it belongs in a di
fferent channel.
   Shows subtle hint: "This might fit better in #engineering.
[Move] [Dismiss]"
   Add to message_send socket handler — run AI classification
asynchronously.

2. MESSAGE MOOD INDICATOR:
   AI analyzes channel sentiment (positive/neutral/negative)
per day.
```

```
   Small mood emoji in channel header: 😊 (positive) 😐 (neut
ral) 😟 (negative)
   Background job runs daily, stores in channel metadata.

3. CONVERSATION INSIGHTS PANEL:
   Right panel option: "📊 Insights" showing for current chan
nel:
   - Messages per day chart (last 30 days)
   - Top contributors
   - Most active hours
   - Common topics (AI-extracted)
   Use existing admin analytics data.

4. SMART MESSAGE SCHEDULING:
   When composing, AI suggests best send time based on:
   - Recipient's timezone
   - When they're usually active
   - Channel's peak activity hours
   Show as hint: "Alice is usually online at 10 AM IST. Sched
ule for then? [Yes]"

5. AUTO-LINK ENRICHMENT:
   When a URL is pasted, auto-fetch Open Graph data:
   - Title, description, image, favicon
   - Show rich link preview card below message
   - For internal links (tasks, pages): show inline preview w
ith status
   Background job using unfurl library.

6. MESSAGE TEMPLATES:
   Users save frequent messages as templates.
   Type "/template" → see saved templates → one-click insert.
   Useful for: standup updates, status reports, common respon
ses.

7. CHANNEL BOOKSHELF:
```

```
   Important channel resources (docs, links, files) pinned as
a "bookshelf" at top.
   Separate from pinned messages — curated resource list.
   Channel admins can add/remove items.

8. QUIET HOURS:
   User sets quiet hours (e.g., 10 PM - 8 AM).
   During quiet hours: all notifications held, delivered as m
orning digest.
   Status auto-sets to DND with moon emoji.
   Stored in user preferences.

9. MESSAGE REACTIONS ANALYTICS:
   In channel insights: "Most reacted messages this week" lea
derboard.
   Shows which messages got the most engagement.
   Helps identify important/popular content.

10. SMART THREAD DETECTION:
    When someone replies to a message without using threads:
    AI detects it's contextually a reply → suggests: "Reply i
n thread to keep the channel organized?"
    Gentle nudge to improve channel hygiene.
```

KEYBOARD SHORTCUTS

```
Cmd+K / Ctrl+K — Search (universal search)
Cmd+Shift+F — Search messages (messaging-specific)
Cmd+N — New message / DM
Cmd+Shift+A — Channel browser
Cmd+[ — Previous channel
Cmd+] — Next channel
Cmd+Shift+S — Toggle sidebar
Cmd+Shift+T — Toggle thread panel
```

```
Cmd+Shift+D — Toggle channel details
Cmd+U — Upload file
Cmd+E — Toggle emoji picker
Cmd+B — Bold text
Cmd+I — Italic text
Cmd+Shift+X — Strikethrough
Cmd+Shift+C — Code block
Escape — Close panel / cancel edit
Up arrow (empty composer) — Edit last message
```

MERGE ALL ROUTERS

Add to `src/server/routers/_app.ts`:

```
import { channelsRouter } from "./channels";
import { messagesRouter } from "./messages";
import { conversationsRouter } from "./conversations";
import { messageSearchRouter } from "./message-search";
import { huddlesRouter } from "./huddles";
import { messagingAdminRouter } from "./messaging-admin";
import { messagingAiRouter } from "./messaging-ai"; // from A
I Features Sprint 7

export const appRouter = router({
  // ... existing routers ...
  channels: channelsRouter,
  messages: messagesRouter,
  conversations: conversationsRouter,
  messageSearch: messageSearchRouter,
  huddles: huddlesRouter,
  messagingAdmin: messagingAdminRouter,
  messagingAi: messagingAiRouter,
});
```

EDGE CASES — SPRINT 4

```
Huddles:
1. Start huddle in channel that already has one → "A huddle i
s already active"
2. Huddle reaches max participants → "Huddle is full"
3. Last person leaves → huddle auto-ends, AI summary posted
4. Network disconnect during huddle → participant removed aft
er 30s
5. Screen share by multiple people → queue, one at a time

Admin:
1. Non-admin tries to export → "Admin access required"
2. Export large channel (100k+ messages) → background job, em
ail download link
3. Retention policy applied → background job deletes old mess
ages, shows warning first

Innovative:
1. Link preview fails (URL unreachable) → show plain URL, no
preview
2. Smart channel suggestion fires too often → limit to 1 sugg
estion per 10 messages per user
3. Mood indicator on channel with <10 messages → don't show
(insufficient data)
```

FINAL VERIFICATION — ALL MESSAGING SPRINTS

```
□ Full message lifecycle: send → edit → delete → all real-tim
e for all channel members
□ Create public channel, private channel, project channel → a
ll work
□ Browse channels → join → star → mute → leave → all work
□ 1:1 DM → group DM → send messages → real-time delivery
```

```
□ Thread: reply in thread → "Also send to channel" → followed
threads sidebar
□ Reactions: add 👍 → count updates → remove → count updates
□ Search: find messages by keyword, filter by user/channel/da
te
□ File upload: drag-drop, paste, 📎 button → inline preview p
er type
□ Notifications: @mention → notified, muted channel → no noti
fication
□ Focus mode → notifications queued → delivered after session
□ Huddle: start → join → leave → AI summary posted
□ AI: summarize thread → catch-up channel → smart replies → t
ranslate
□ Admin: view analytics → export channel → set retention → ma
nage members
□ Keyboard shortcuts all work
□ Mobile responsive: sidebar collapses, single-column layout
□ Typing indicator: shows "Alice is typing..." → auto-clears
after 5s
□ Presence: online (green) → away (yellow) → DND (red) → offl
ine (gray)
□ Unread badges accurate across all channels
□ No TypeScript errors: npx tsc --noEmit → clean
```

MESSAGING SYSTEM COMPLETE.